FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Joshua Emmanuel A. Guerrero John Patrick B. Villaraza
Section:	AN41
Professor:	Crisola G. Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base

2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

```
facts = {
   "temperature": 32,      # Category 1: Temperature
   "humidity": 85,         # Category 1: Temperature/Environment
```

```
    "is_raining": True,     # Category 2: Status
    "is_cloudy": True,      # Category 2: Status
    "user_present": True    # Category 3: Presence
}

def rule_hot(facts):
    if facts.get("temperature", 0) > 30 and facts.get("user_present", False):
        return "Turn on the aircon"
    return None

def rule_rain(facts):
    if facts.get("is_raining", False):
        return "Bring umbrella"
    return None

def rule_dehumidifier(facts):
    if facts.get("humidity", 0) > 80:
        return "Turn on dehumidifier"
    return None


def run_rbr_inference_engine(facts_db, rules_list):
    inferred_actions = []
    for rule in rules_list:
        action = rule(facts_db)
        if action:
            inferred_actions.append(action)

    print("Rule-Based Reasoning Actions:")
    for action in inferred_actions:
        print(f"- {action}")
    return inferred_actions

# Execute Demonstration
rules = [rule_hot, rule_rain, rule_dehumidifier]
run_rbr_inference_engine(facts, rules)
print("\n" + "="*40 + "\n")


case_base = [
    {
        "problem": {"won't turn on": True, "noise": False},
        "solution": "Check power cable."
```

```
    },
    {
        "problem": {"won't turn on": True, "noise": True},
        "solution": "Inspect fan motor for blockage."
    },
    {
        "problem": {"won't turn on": False, "noise": True},
        "solution": "Clean or replace fan bearings."
    }
]


def calculate_similarity(new_problem, case_problem):
    score = 0
    total_keys = set(new_problem.keys()).union(set(case_problem.keys()))
    for key in total_keys:
        if new_problem.get(key) == case_problem.get(key):
            score += 1
    return score / len(total_keys) if total_keys else 0.0


def retrieve_and_reuse(new_problem, case_base):
    best_match = None
    highest_score = -1.0

    for case in case_base:
        sim = calculate_similarity(new_problem, case["problem"])
        if sim > highest_score:
            highest_score = sim
            best_match = case

    print(f"[Retrieve & Reuse] Most similar case score: {highest_score * 100:.1f}%")
    return best_match["solution"] if best_match else "No solution found."


def revise_solution(proposed_solution, manual_adjustment=None):
    if manual_adjustment:
        print(f"[Revise] Solution manually updated from '{proposed_solution}' to '{manual_adjustment}'")
        return manual_adjustment
    print(f"[Revise] Proposed solution accepted without changes: '{proposed_solution}'")
    return proposed_solution
```

```
def retain_case(case_base, new_problem, final_solution):
    new_case = {
        "problem": new_problem,
        "solution": final_solution
    }
    case_base.append(new_case)
    print(f"[Retain] New case successfully learned and stored into Case Base.")


new_user_problem = {"won't turn on": True, "noise": False}
print(f"New Problem Input: {new_user_problem}")

# 1. Retrieve & Reuse
retrieved_sol = retrieve_and_reuse(new_user_problem, case_base)

# 2. Revise (Optionally pass custom string to revise)
final_sol = revise_solution(retrieved_sol, manual_adjustment="Check power cable and fuse box.")

# 3. Retain
retain_case(case_base, new_user_problem, final_sol)
```